



CONTENT

INTRODUCTION	3
DESCRIPTION	3
TASK	5

INTRODUCTION

Bugatti Rimac is constantly searching for new team members in Embedded Software Development department. To be able to pick the best candidates we prepared exam that will help us in that task. From your solution, we want to see how you think and how good are your engineering skills. Please read the whole document first and then start with creating a solution. If something is not defined in text or pictures of this exam, you are allowed to have assumptions but we are kindly asking to note them in your solution. Task solution is not designed for any particular programming language, but our preference is for it to be delivered in Matlab/Simulink (.slx files, .m files, .mat files, various parameters files should be part of the same project in the same folder) OR in a Classical Embedded C (simple project structure with organization of source code, header files and other dependencies as in a typical embedded ECU project repository). For C implementations, the solution should demonstrate equivalent architectural concepts as would be expected in a model-based solution. This may include state machines, configurable relay instances, scheduler integration, reusable modules, parameterization through configuration structures, and clear separation between application logic and hardware abstraction.

This is a conceptual task where the emphasis on the solution evaluation is on the presented system – level concept, rather than the pure code itself. For C-based solutions, code completeness is less important than demonstrating sound software architecture, modularity, scalability, readability, maintainability, and embedded-system design principles.

DESCRIPTION

We will assume the system is operating and monitoring one or multiple relays, depending on system desired functionality, and user requirements. What these relays are used for is not relevant for this task, but we can assume they are used for powering some high voltage devices of irrelevant form and functionality. System is equipped with multiple digital inputs (DI) and digital power outputs (DPO). Digital outputs can be set to 12V (high level) or to GND (low level) using the software control of the outputs. There is also indefinite number of digital input ports and pins. Controller can only detect/read “high” or “low” input states or levels. There is no way to detect actual voltage value with controller. We can assume that there is some input voltage threshold between “high” and “low” pin state, somewhere around middle of the Vdd voltage. Relays feedback output voltage is mostly at stable voltage, but there is occasional feedback signal level bouncing which may be caused by physics of contactors closing and opening. Feedback signal amplitude voltage range is between 0V and 12V. We can know for sure that time needed for stable read of the relay feedback after

changing its state is maximum 30ms. Candidates may assume that software is executed periodically by a scheduler (e.g. 1ms, 5ms or 10ms task period) and may implement timing supervision using counters, timers, state-machine timeouts or equivalent mechanisms. Time needed for DPO to change its state to stable constant is 1ms.

Relays can be “normally open” (NO) and “normally closed (NC). If relays are normally open that means that if you don’t apply voltage to them, they should stay (or change to) in open state. If they would be normally closed it would be another way around. Relay control lines are connected to some digital power output (DPO) of the controller, open or close the relay by engaging the main relay contactor coil. Regarding feedback lines of the relays, they are connected to one of the digital inputs (DI) of the system. Using this digital input we can detect (through relay feedback contactor) in which state relay is (open or closed). There can be logical “high” or “low” feedback state detected. Relay feedback contactor is following relay main contactor. That is, if main (power) contactor is closed, feedback contactor is closing at the same time, and vice versa. High voltage and low voltage parts of the relay are completely isolated, and high voltage part is not relevant for this task.

Also, these relays may get damaged if operated while in high current loads, so we need to monitor their feedback status, and react just in time. We can say that relay is welded if we are trying to open the relay contactor and its feedback says that it is still closed. Also, we can say that relay is constantly open if we are trying to close it and its feedback says it remains open even after some short time has passed.

Image below shows one relay connected to DPO and DI pair 1. Next, second relay would/should be connected to next pair (DI_2 and DPO_2), etc.

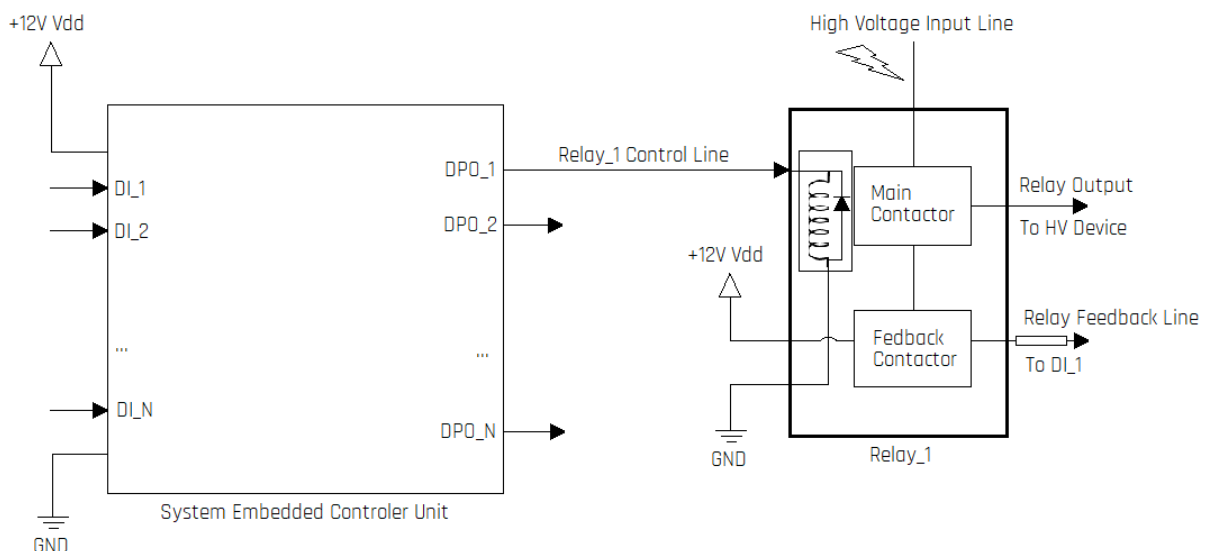


Figure 1 - Example of System Schematic

TASK

The focus of this task is the relay controller itself. Controller should be able to read and serve the request, as well to detect and react to faulty relays. Relay requests (to open and close relays) and feedback levels (“low” or “high”) are coming from internally connected signals (values can be created or generated randomly and periodically) and simulated all together. Requests at signal source or feedback input can be also connected as “Simulink constant block” or can be represented by a defined value. Solution (simulation) should have 2 (or 3 if preferable) relays connected. There should be at least one NO and one NC relay. User should be able to define/config the type of relay **for each input/output pair used** simply via representation of a calibratable parameter (e.g. Simulink parameter or constant). For C implementations, calibratable parameters may be represented through either:

- configuration structures,
- constant configuration tables,
- initialization data,
- compile-time configuration files,
- or equivalent embedded software configuration mechanisms.

Feel free to use multiple subsystems/modules to define each input/output pair used for each relay in the system. For Simulink solutions, a configurable library block is preferred. For C solutions, a reusable software module/component with configurable relay instances is preferred.

This controller is part of real-time embedded system, and it can monitor the relevant time passed. It is necessary to serve the open/close request, and to monitor the relays status and relays requests constantly (open/closed, error/no-error, welded/constantly open). If there is relay error detected, the controller should transition and stay in error state (until controller restart), and it should immediately **open** all the relays and prohibit further relay closing request (until controller restart). The mechanism used to represent the controller state is implementation-dependent and may be realized through a state machine, software component state variable, Stateflow chart, enum-based implementation, or equivalent architecture.

Design and implement software unit/module that:

- Controls (opens or closes) the relay over the digital power output of the system reacting to internal (hardcoded or simulated) requests.
- Continuously monitors and reports relay status by reading its feedback status via digital inputs.
- Detects errors by recognizing a mismatch between relays control and feedback line.

- Each relay type should be easily configurable in the controller (or outside parameter) itself and saved as a library block.
- Provide status (open/closed) of relays to the rest of the system.
- Provide error status (state) of relays to the rest of the system (no error/welded/constantly open).
- Since this is a real-time embedded system running on an MCU, some form of scheduler or time-controlled execution is mandatory. For Simulink solutions this may be represented through model timing and execution rates. For C solutions this may be represented through a periodic task, cyclic executive, RTOS task, timer-driven execution, or equivalent scheduling concept. Timing supervision for relay feedback validation should be clearly visible in the design.
- Have documentation in terms of flow and state diagram for task execution part of the unit/module. For C implementations, UML state diagrams, flowcharts, software architecture diagrams, module decomposition diagrams, or equivalent representations are acceptable.
- Controller and its internal entities shall be designed for reuse. For Simulink solutions this may be realized through reusable library blocks. For C solutions this may be realized through reusable modules, components, libraries, or object-like software instances.
- Min number of relays connected (simulated) in the system is 2 – at least 1 NC and 1 NO relay
- Limit your relay number in the model to max of 10
- Simulated “Scope”/output screenshot is optional bonus
- Complete task package should include one pdf file with flow diagram, state diagram, task solution in text and pictures/diagrams/schematics form and **C project repository** (source files, header files, config data, *build instructions (OPTIONAL)*, architecture overview with state diagram and flow diagram, explanation of scheduling/timing concept) or **Simulink project repository** (.slx files with optional .m and .mat parameters/functions/scripts files).